# A Comprehensive Survey of World Models for Coding

Keon Kim[1]

[1]Independent Researcher

Code & Survey  arXiv

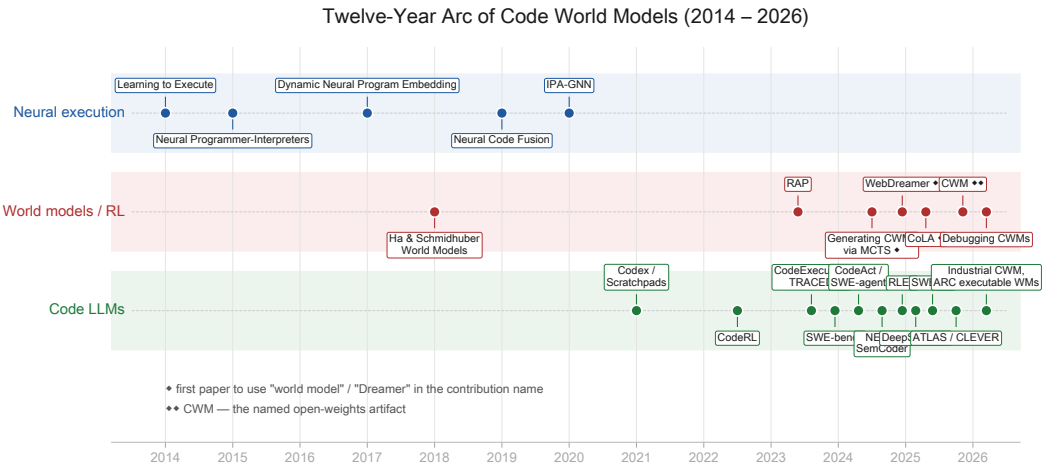


Figure 1: Twelve-year arc of code world models. Three parallel lineages — neural execution, world models and RL, and code LLMs — converge in 2025–2026 around the named artifact CWM. Single-diamond markers in the figure tag the first paper to use "world model" or "Dreamer" in a contribution name; the double diamond marks CWM as the named open-weights artifact.

## 0.1 Abstract

A *world model* is an internal predictor over environment dynamics, used to imagine the consequences of actions. In coding, the environment is the program: its runtime state, execution trace, filesystem, tests, and developer task. Zaremba and Sutskever (Zaremba and Sutskever, 2014) trained an LSTM to predict Python execution output in 2014. Meta FAIR released CWM (FAIR CodeGen Team et al., 2025), the first open-weights LLM branded a Code World Model, in October 2025. Internal models of execution can be learned. Whether they are necessary, architecturally separable, or causally responsible for coding-agent gains remains unestablished.

This survey synthesizes 184 papers spanning four research lineages: neural execution, trace pretraining and the CWM line, agentic software engineering with execution feedback, and verifier-grounded code generation. It traces the twelve-year arc, builds a three-axis taxonomy (functionality, temporal, representation), produces system cards for thirteen representative systems, assembles protocol-stratified tables for SWE-bench, CRUXEval, web agents, and formal verification, develops seven critical theses, and lists open problems. The empirical evidence supports execution-grounded supervision as a reliable lever for code-agent gains on runtime-reasoning benchmarks. Open questions concern which architectural commitments — token-space prediction, latent dynamics, or synthesized simulators — best capture program semantics.

Preprint.

## 0.2 Theses Summary

The seven critical theses developed in §17, each grounded in a specific disconfirmation from the corpus:

1. The architectural design space for code WMs has three concrete forms. Forward state prediction admits a clean tripartition (§3.1): neural dynamics models in compressed latent space (N1, no code exemplar yet), latent-action models such as CoLA (N2), and synthesized executable simulators such as GIF-MCTS and ARC executable WMs (N3). Most "Code World Model" systems today (CWM, TRACED, SemCoder, NExT) extend the LLM training distribution without committing to any of these three forms; whether the architectural commitment pays off remains the central open empirical question.
2. Trace pretraining has a causal-isolation problem. The ablation study Do Code Semantics Help? (Wang et al., 2025a) finds no trace representation consistently improves code synthesis across multiple backbones. CWM's 65.8% SWE-bench Verified result confounds trace mid-training, ForagerAgent trajectories, and RL.
3. The Dreamer-for-code gap may be a non-problem. Vision required latent rollouts because pixels are expensive. Python state is small and CPython is free, so the architectural pressure does not transfer. CWM in token space reaches 65.8% on SWE-bench Verified at 32B parameters.
4. PRMs and forward WMs solve complementary problems. Process reward models score partial trajectories; forward WMs predict next states. The design space for combining them — a PRM critic over rollouts from a forward predictor — is largely unexplored in code, despite being standard in model-based RL.
5. General Agents Contain World Models (Richens et al., 2025) is weaker than its title. The theorem applies under restrictive assumptions (full observability, stationarity, deep-composite-goal regret bounds) that SWE agents demonstrably violate.
6. The verifier-grounded lineage leads on a narrow scope. ATLAS, Re:Form, CLEVER, VeriStruct, and AutoRocq produce machine-checkable correctness. They lead on synthesis-from-spec (e.g., ATLAS DafnySynthesis 65.8% pass@5); they do *not* lead on end-to-end verified codegen from natural language (CLEVER ?1/161 Lean).
7. The evaluation gap is structural. No benchmark holds policy fixed and varies WM quality. Models with 85–98% output-prediction accuracy still produce traces with systematic errors throughout. Outcome accuracy decouples from process fidelity.

---

## 0.3 1. Introduction

Autoregressive code LLMs generate tokens conditioned on syntactic context. Correct programs are objects in two domains: source tokens, and the values, control flow, side effects, and developer intent those tokens encode. The world-model framing comes from model-based reinforcement learning, where it names an internal predictor of environment dynamics. The hypothesis examined here: training on execution rather than only on source-token prediction reduces the gap between code that compiles and code that runs.

Two adjacent surveys cover non-overlapping ground. A Survey on LLMs for Code Generation (Jiang et al., 2024) maps the code-LLM space without the world-model lens. Understanding World or Predicting Future (Ding et al., 2024a) maps world models in general without the code lens. A Comprehensive Survey on World Models for Embodied AI (Li et al., 2025b) maps embodied world models but excludes the coding domain. The intersection, the subject of this document, has cohered only recently into a recognizable program.

---

## 0.4 2. Methodology

The corpus contains 184 arxiv preprints, assembled in four passes (March–May 2026) seeded from CWM (FAIR CodeGen Team et al., 2025) via citation BFS and targeted topic searches. A

paper enters the corpus if it intersects both world-model or state-tracking architectures and code generation, debugging, repair, or agentic coding. Pure vision world models (DreamerV1–V3, V-JEPA, Genie) and pure code-LLM papers without a world-model angle are excluded except as cited precedent. Date cutoff: 2026-05-15. One curator performed the taxonomy coding. Numerical claims in §§6–14 derive from abstracts, §16 from source PDFs. 60% of the corpus dates from 2025 or later.

Cross-tabulation across the two primary taxonomic axes:

| | What is modeled | | | |
|---|---|---|---|---|
| Architectural form (§3.1) | Variable values / traces | Env / repo state | Spec / intent | Adversarial |
| Definition D (data-side) | ~70 | ~30 | ~12 | ~6 |
| N1 (neural dynamics) | 0 | 0 | 0 | 0 |
| N2 (latent action) | 2 | 1 | 0 | 0 |
| N3 (synthesized simulator) | 4 | 2 | 1 | 0 |
| Verifier / PRM (related) | ~15 | ~10 | ~10 | ~5 |

Two facts dominate the table. First, the descriptive bucket (D) holds the majority of the corpus and concentrates on variable values and traces. Second, the N1 row contains zero code exemplars: no system in the corpus instantiates a Dreamer-style learned dynamics model for code. §18 lists this as an open problem.

---

## 0.5 3. Defining a World Model for Coding

Three properties distinguish a *code* world model from a generic one and explain why the design space for coding diverges from the vision-WM design space.

- Executable ground truth. The environment ships with a precise simulator: CPython, the Java VM, an SMT solver, a hardware verifier. The WM can be evaluated against this oracle at training time, which is rare for vision and impossible for natural-language reasoning.
- Compact, observable state. A program frame at a given line is a small structured object: local variables, the call stack, file handles. Pixel-space WMs needed compression because a frame is high-dimensional. Code WMs face the opposite question — whether to predict full state or to abstract it.
- Source–trace duality. Every executed trace corresponds to a piece of source, and every piece of source admits many traces. The two views are linked by the interpreter, so a WM for coding can be trained on either side or on their alignment.

These three properties explain why most current code WMs are token-space predictors over execution traces rather than latent-space rollouts: the simulator is free, state is small, and source–trace alignment supplies plentiful supervised pairs. They also explain why verifier-grounded systems form a distinct cluster: the ground-truth oracle replaces the learned WM entirely.

The literature uses *world model* in two distinct senses, which this survey separates.

### 0.5.1 3.1 Two definitions

Definition D (descriptive, permissive). A *world model for coding* is any code LLM whose training objective concretely encodes program semantics — execution traces, runtime state, envi-

ronment feedback, or simulated outcomes — in addition to or instead of source-token prediction. This matches the field's current usage. Under D, CWM, TRACED, SemCoder, LLM-JEPA, DyMo, RLEF, and most papers in the corpus qualify as world models.

Definition N (normative, strict). A *world model for coding* is a system with an explicit, separable forward-prediction mechanism. N splits into three sub-types:

- N1 — Neural dynamics model. A learned `W : (state, action) → next_state` instantiated as a distinct architectural component with latent recurrent state, separate dynamics head, or inverse model. Dreamer-class. No code exemplar in the corpus.
- N2 — Latent-action model. A learned action abstraction over a base LLM, with the LLM serving as transition model in compressed action space, supporting rollout or tree search over latent actions. CoLA (Jia et al., 2025) provides the canonical example.
- N3 — Synthesized executable simulator. The world model is an executable program (Python, DSL) synthesized by an LLM and run against ground-truth transitions during planning. GIF-MCTS (Dainese et al., 2024), WorldCoder, Executable WMs for ARC-AGI-3 (Rodionov, 2026).

Each N-subtype carries a distinct architectural commitment. N1 commits to learned latent dynamics; N2 commits to a discrete latent-action space; N3 commits to program synthesis as the dynamics. Conflating them, as the loose Dreamer-for-LLMs framing did, obscures which architectural bet a system makes. Under any N-subtype, CWM, TRACED, SemCoder, and most of the corpus do *not* qualify.

The two definitions agree on the empirical fact that execution-grounded supervision helps. They disagree on whether that grounding warrants the name *world model* in the model-based-RL sense.

| Aspect | Definition D (descriptive) | Definition N (normative) |
|---|---|---|
| Granted to | Any LLM trained with execution-related signal | Only systems with explicit forward-prediction module |
| Includes CWM | Yes | No (architecturally a Llama-class decoder) |
| Includes TRACED, SemCoder, NExT | Yes | No (auxiliary heads on a Transformer) |
| Includes CoLA, GIF-MCTS | Yes | Yes |
| Includes PRMs (ExecVerify, ThinkPRM) | Often grouped | No (critics, not forward predictors) |
| Includes verifiers (Lean, Dafny, Z3) | Sometimes grouped | No (deterministic oracles, not learned WMs) |
| Includes Dreamer / V-JEPA (vision precedents) | Yes | Yes |

This survey uses D throughout the catalog sections (§§6–16) because that reflects how the literature talks, and N in §17 because the critical synthesis depends on it. The switch is marked each time it matters. The two-definition framework resolves the tension between field usage and architectural precision.

### 0.5.2 3.2 What is modeled

Orthogonal to D-vs-N, *what* a system models varies: variable values and stack frames (CWM, CodeExecutor); execution traces (NExT, SemCoder, TRACED); test outcomes (LEVER, RLEF); OS or web environment state (WebDreamer, Dyna-Think); repository state (RepoGraph); spec or developer intent (ATLAS, Re:Form); adversarial behavior (Double Life of CWMs). A system typically commits strongly to one or two of these.

CWM occupies an instructive position: behaviorally explicit (it emits stack frames) but architecturally implicit (a 32B Llama-class decoder with no separate dynamics head). SemCoder, NExT,

and TRACED follow the same pattern. The explicit-WM label flattens this distinction; §17.1 develops the consequence.

---

## 0.6 4. Twelve Years of Code World Models

The lineage reads as a sequence of inheriting questions. Each era's answer dissolved the previous era's bottleneck and exposed the next.

(See Figure 1 at the start of the paper for the timeline diagram.)

Diamond markers in the figure tag the moments where *world model* enters the name of the contribution.

Pre-2020: can a network execute code? Learning to Execute (Zaremba and Sutskever, 2014) trained an LSTM to predict Python output on bounded-loop programs. Neural Programmer-Interpreters (Reed and de Freitas, 2015) and Differentiable Forth built differentiable program counters. Dynamic Neural Program Embedding (Wang et al., 2017) embedded real interpreter traces. Neural Code Fusion (Shi et al., 2019) and IPA-GNN (Bieber et al., 2020) used GNN attention as a program counter. By 2020 the answer was yes, but only with the interpreter encoded into architecture, which did not scale to real languages. Ha & Schmidhuber (Ha and Schmidhuber, 2018) had named the pattern for vision-RL; the coding community had not yet borrowed it.

2020–2022: can the training include execution? Codex (Chen et al., 2021) and MBPP (Austin et al., 2021) made code generation a target. Scratchpads (Nye et al., 2021) showed a Transformer could predict program output when allowed to emit intermediate computation first, the Dynamic-NPE move at LLM scale, without architectural surgery.

2023: trace pretraining as a named recipe. CodeExecutor (Liu et al., 2023) trained a Transformer to emit per-line state traces from source. TRACED (Ding et al., 2023) generalized this to a pretraining auxiliary. CRUXEval (Gu et al., 2024a) provided the eval that became standard. In parallel, Reflexion (Shinn et al., 2023) and Self-Debug (Chen et al., 2023) used execution feedback in-context, LEVER (Ni et al., 2023) used it during decoding, and RAP (Hao et al., 2023) ran MCTS over an LLM-as-world-model.

2024: from models that simulate to agents that act. SWE-bench (Jimenez et al., 2023) shifted the task to real GitHub repos. CodeAct (Wang et al., 2024b), SWE-agent (Yang et al., 2024), NExT (Ni et al., 2024), RLEF (Gehring et al., 2024), WebDreamer (Gu et al., 2024b), and Generating Code World Models via MCTS ((Dainese et al., 2024), source of the name) define the agentic turn. The WM moved from weights to loop.

2025: the CWM artifact. DeepSeek-R1 (DeepSeek-AI et al., 2025) and SWE-RL (Wei et al., 2025d) scaled reasoning-RL. CoLA (Jia et al., 2025) and LLM-JEPA (Huang et al., 2025) ported Dreamer- and JEPA-style ideas to LLMs. General Agents Contain World Models (Richens et al., 2025) provided the existence theorem. CWM (FAIR CodeGen Team et al., 2025), 32B open-weights, mid-trained on 5T tokens of Python traces plus 3M ForagerAgent trajectories, made the artifact downloadable.

2026: stress-testing and broadening. Debugging CWMs (Rahmani, 2026) and Demystifying Errors in LLM Reasoning Traces (Abdollahi et al., 2025) catalog failures. Industrial CWM (Yang et al., 2026b), Parallel-Code WMs (Singh et al., 2026), and Executable WMs for ARC-AGI-3 (Rodionov, 2026) generalize the recipe. Reinforcement World Model Learning (Yu et al., 2026) trains the WM rather than the policy.

---

## 0.7 5. Taxonomy: Three Axes of Code World Models

Two cuts at the taxonomy prove useful, and they complement each other. The first, a *lineage* cut, asks which research thread produced the system, and structures §§6–13. The second, more

durable cut adapts the three-axis framework of Li et al. (Li et al., 2025b) to the code domain. The lineage map follows; the three axes appear in §§5.1–5.3.

| Representation | Vision analog | Code-WM exemplars |
|---|---|---|
| Token Sequence (TS) | Token-as-pixel (IRIS, Genie, Sora) | CWM, CodeExecutor, TRACED, NExT, SemCoder |
| Synthesized executable (N3) | — | GIF-MCTS, WorldCoder, ARC executable WMs |
| Global Latent Vector (GLV) | RSSM (Dreamer V1–V3) | white space — no code exemplar |
| Spatial / Structural Grid (SLG) | OccWorld, DriveWorld (BEV/voxel) | white space — no code exemplar |
| Decomposed Object / Slot (DOR) | SlotFormer, object-centric WMs | white space — no code exemplar |

Verifiers (Lean / Dafny / Z3) and PRMs are intentionally absent: a verifier is a grounding oracle, not a representation; a PRM is a critic, not a forward predictor.

Figure 2: Representation taxonomy of code world models. Each row is one representation class (left), its vision precedent (centre), and the code-WM systems that instantiate it (right). Three classes — Global Latent Vector (Dreamer-style), Spatial/Structural Grid, and Decomposed Object/Slot — have no code exemplar in the corpus and are the survey's white-space gaps.

Adjacent WM surveys converge on overlapping splits that the three-axis framework subsumes as projections. Ding et al. (Ding et al., 2024a) split top-level by *implicit representation* vs *future prediction*. JiahuaDong's awesome-list organizes by *paradigm*: RL-based, observation-generative, latent-space, object-centric. knightnemo's list surfaces *pixel vs mesh vs latent* as cross-cutting tags. The three axes — functionality, temporal modeling, and representation — capture the choices a system makes regardless of which lineage it belongs to.

### 0.7.1 5.1 Axis 1 — Functionality

- Decision-coupled WMs model only the slice of the world relevant to acting on it. CWM (FAIR CodeGen Team et al., 2025), RLEF (Gehring et al., 2024), and WebDreamer (Gu et al., 2024b) are decision-coupled. Their WMs exist to enable code generation, RL planning, or web navigation respectively. CWM does not predict global filesystem state; it predicts the next Python frame because the next action depends on it.
- General-purpose WMs model the environment without reference to a particular task. The general-agents-contain-world-models theorem (Richens et al., 2025) provides the abstract limit. In the corpus, only the largest CWM-class models with broad mid-training approach generality; most code world models are decision-coupled to a subtask (repair, completion, agent control).

### 0.7.2 5.2 Axis 2 — Temporal Modeling

- Sequential simulation/inference. Step-by-step autoregressive rollout. CWM, NExT, SemCoder, all the trace-pretraining systems, and most LLM-as-WM planners (RAP, WebDreamer in its MPC loop) live here. The state updates one timestep at a time. The vision analog is RSSM (Hafner et al., DreamerV1–V3).
- Global difference prediction. Predict the entire future state at once, in parallel. The vision analog is video-diffusion or masked-JEPA. In code, this fits diffusion code models (DiffuCoder, (Gong et al., 2025); Dream-Coder 7B, (Xie et al., 2025)), where the next state is sampled jointly rather than autoregressively, and the specification-is-the-program framing (Monperrus, 2026), where the entire trace is the spec.

- Static, no-trace. Some systems (SemCoder's static mode, the trace-free baselines in Do Code Semantics Help?) explicitly drop temporal modeling at inference, reducing to single-shot prediction.

### 0.7.3 5.3 Axis 3 — Representation

The WM literature has converged most strongly on this axis, and the code-WM literature remains most uneven on it. Adapting Li et al.'s four-category split (GLV / TFS / SLG / DRR) to code yields five classes, of which only two are well-populated.

| Class | Encodes the world as | Vision analog | Code exemplars |
|---|---|---|---|
| Token Sequence (TS) | Discrete or continuous token streams with execution traces, variable bindings, or rationales interleaved with source | Token-as-pixel (IRIS, TWM, Genie, Sora) | CWM (FAIR CodeGen Team et al., 2025), CodeExecutor (Liu et al., 2023), TRACED (Ding et al., 2023), NExT (Ni et al., 2024), SemCoder (Ding et al., 2024b) — the dominant code-WM mode |
| Global Latent Vector (GLV) | A compact vector updated recurrently, encoding the entire program/agent state | RSSM (Hafner et al., DreamerV1–V3) | No clean exemplar. CoLA (Jia et al., 2025) introduces a learned action codebook but is otherwise a standard LLM, not RSSM-style |
| Spatial / Structural Grid (SLG) | A geometric or structural grid (BEV/voxel in vision; AST, call-graph, CFG in code) | OccWorld, DriveWorld | No exemplar. RepoGraph (Ouyang et al., 2024) uses a static dependency graph but does not predict over it as a WM |
| Decomposed Object / Slot (DOR) | Distinct persistent latent slots for objects in the world | SlotFormer and object-centric WMs | No exemplar. No code-WM models variables, scopes, or classes as discrete persistent slots |
| Synthesized executable (N3) | The world model *is* an executable program, synthesized rather than learned | (orthogonal to vision) | GIF-MCTS (Dainese et al., 2024), WorldCoder, Executable WMs for ARC-AGI-3 (Rodionov, 2026) |

Verifiers (Lean, Dafny, Z3) and PRMs are intentionally absent from this table. A verifier is not a *representation* of the world; it is a grounding oracle over candidate artifacts. A PRM is a backward-looking critic, not a forward predictor. Both belong in §5.4 below.

Three white spaces. The Dreamer-style GLV, the object-centric DOR, and the spatial-grid SLG representations have not been instantiated for code, despite being mature for vision. §18 lists these as open problems, with the caveat that not all three look equally promising. The Dreamer-style gap is contestable (§17.3), while the object-centric and structural-grid gaps look more genuine.

#### 0.7.4 5.4 Grounding mode (orthogonal to representation)

A second classification asks how each system's predictions are validated. This analog of Li et al.'s reality column addresses the decoupling between WM-head accuracy and downstream task success that DyMo (Guo et al., 2025) and §17.7 develop.

| Grounding mode | Definition | Code exemplars |
|---|---|---|
| None / self-report | Predictions never validated against ground truth | RAP (Hao et al., 2023), basic LLM-as-WM planners |
| Execution-grounded | Predictions checked against real interpreter / runtime | CWM (FAIR CodeGen Team et al., 2025), TRACED (Ding et al., 2023), NExT (Ni et al., 2024), SemCoder (Ding et al., 2024b), RLEF (Gehring et al., 2024) |
| Verifier-grounded | Outputs checked by Lean / Dafny / Verus / Rocq / Z3 | ATLAS (Baksys et al., 2025), Re:Form (Yan et al., 2025), CLEVER (Thakur et al., 2025), VeriStruct (Sun et al., 2025), AutoRocq (Tu et al., 2025) |
| Synthesized-simulator-checked | Synthesized world model checked against held-out transitions | GIF-MCTS (Dainese et al., 2024), Executable WMs for ARC-AGI-3 (Rodionov, 2026) |
| Critic-grounded (not a WM) | Backward-looking value/quality score, no rollout | ExecVerify (Tang et al., 2026), SWE-PRM (Gandhi et al., 2025), ThinkPRM (Khalifa et al., 2025) — listed for contrast; these are critics, not WMs (§17.4) |

Grounding mode is orthogonal to the representation axis: a token-sequence representation can be execution-grounded (CWM) or self-report (RAP); an N3 synthesized-simulator can be checked against transitions (GIF-MCTS) or never validated. The WM-fidelity question of §17.7 structurally asks whether a system's grounding mode is strong enough to falsify a bad WM at training time.

---

### 0.8 6. Foundations: Neural Execution as Implicit World Modeling

Learning to Execute (Zaremba & Sutskever, (Zaremba and Sutskever, 2014)) established both feasibility and brittleness. Show Your Work — Scratchpads (Nye et al., 2021) provided the pivotal contribution: by training a Transformer to emit intermediate computation states, the authors recovered much of the LSTM-era execution-prediction performance at scale, presaging the trace-pretraining lineage of §6. CRUXEval (Gu et al., 2024a) and REval (Chen et al., 2024) provide the canonical execution-reasoning benchmarks. The lesson the field absorbed: *replacing* the interpreter with a neural network is harder than *augmenting* a transformer with interpreter-style supervision. Modern systems all take the latter path.

---

### 0.9 7. The Trace-Pretraining and CWM Lineage

#### 0.9.1 7.1 Trace-pretraining as a recipe

CodeExecutor (Liu et al., 2023) trains a Transformer to simulate Python execution token-by-token. TRACED (Ding et al., 2023) adds dynamic-state supervision to a code-LLM pretraining

mix. NExT (Ni et al., 2024) formats traces as natural-language rationales, letting a chat-style LLM reason about runtime behavior via chain-of-thought. SemCoder (Ding et al., 2024b) generalizes to monologue reasoning linking source-text to execution state.

The 2025 wave consolidated and stress-tested the approach. What I cannot execute, I do not understand (Armengol-Estapé et al., 2025) trains and evaluates LLMs explicitly on traces with dynamic scratchpads, pushing Llama-3.1-8B from 37.8% to ~80% on CRUXEval-O. Code Execution as Grounded Supervision (Jung et al., 2025) repurposes line-by-line traces as verifiable CoT. Self-Execution Simulation (Maimon et al., 2026) lets the model train on its own execution predictions. Demystifying Errors in LLM Reasoning Traces (Abdollahi et al., 2025) audits where trace-trained LLMs fail. Do Code Semantics Help? (Wang et al., 2025a) provides the most damaging paper in the lineage: a comprehensive ablation across DeepSeek-Coder, LLaMA-3, and Gemma-2 with five trace representations found that no single representation consistently improved code generation, and in 7 of 9 synthesis settings the no-trace baseline won or tied.

#### 0.9.2 7.2 TRACED (Ding et al., 2023)

TRACED augments a RoBERTa/UnixCoder pre-training mix with two execution-grounded heads on top of standard MLM: per-line program-state classification (variable type and quantized value over 30 bins) and per-line execution coverage. Trained on ~121k C traces from CodeNet collected via gdb, it showed that quantized variable-value prediction works as an auxiliary signal; concrete values lose to discretized bins. On static execution estimation, full-path accuracy rose from UnixCoder's 63.7% to 71.6%, and downstream clone retrieval and defect detection improved modestly. The contribution: trace prediction as a pre-training side objective, not a separate model.

#### 0.9.3 7.3 NExT (Ni et al., 2024)

NExT inlines execution traces into source as Python-style comments (`# (k) varA=...; varB=...`) and trains PaLM 2-L on (rationale, fix) candidates via STaR-style self-training: sample 32 candidates per problem, accept those that passed unit tests, SFT on the accepted set, repeat. After ten iterations Mbpp-R pass@1 climbed from 23.2% to 49.3% (+26.1 absolute). The result that matters most for the rest of the survey: NExT retains a +17 absolute gain (23 → 40.8) on Mbpp-R when traces are removed at inference, which makes it the clearest example of trace pretraining transferring to non-trace inference on a repair task.

#### 0.9.4 7.4 SemCoder (Ding et al., 2024b)

SemCoder formalizes monologue reasoning linking four code modalities — natural-language description, source, operational trace, and abstract input-invariant constraints — under a single NTP objective with rejection-sampled training data (the PYX corpus). Distinctive features: forward and backward monologues (NExT is forward-only), abstract-semantics constraints rather than concrete state at every step, and entirely static inference. SemCoder-1.3B reached CRUXEval-I/O 63.6/65.1 vs GPT-3.5-turbo's 50.3/59.0, and a monologue-format ablation beat Scratchpad and NExT.

#### 0.9.5 7.5 CWM (FAIR CodeGen Team et al., 2025)

CWM is a 32B dense decoder-only Transformer with grouped-query attention, sliding-window blocks, and RoPE — architecturally Llama-class. What earns it the world-model name is the mid-training datamix: 5T tokens of Python observation-action traces (120M traced functions, 262k CodeContests traces, 70k repo-level traced commits, 75M natural-language rewrites) plus 3M ForagerAgent SWE trajectories from 10.2k Dockerized repositories. Tokens are formatted so next-token prediction *is* next-state prediction at line granularity. CWM reached 65.8% on SWE-bench Verified with test-time scaling (best@16 over 40 verifier-reranked samples) and 94.3% on CRUXEval-Output. Its second technical contribution is Activ, which uses GitHub Actions CI to scale executable repository images. CWM is behaviorally explicit (emits stack frames) but architecturally implicit (no separate dynamics head).

Important caveat (developed further in §17): the 65.8% headline is not pure pass@1 but best-of-16 with verifier reranking. Pure pass@1 is approximately 53–55%. The trace-mid-training

contribution is not causally isolated from the ForagerAgent-trajectory contribution and from the joint-RL contribution. Without an ablation removing one while holding the others fixed, the world-model component's causal role remains unfalsifiable.

#### 0.9.6 7.7 GIF-MCTS / Generating Code World Models via MCTS (Dainese et al., 2024)

GIF-MCTS treats the world model itself as a Python program: an `Environment.step(s,a) → (s', r, done)` class synthesized by an LLM to match a small batch of pre-collected `(s, a, r, s', d)` transitions. MCTS over partial programs uses three action types (*generate* lines, *improve* full program given a failing transition, *fix* runtime/syntax errors), with reward equal to the fraction of transitions reproduced correctly. The synthesized world model, once compiled, runs 4–6 orders of magnitude faster than calling an LLM as world model. On APPS-Competition it reached 28.3% strict pass@20 (Llama-3-70B), beating WorldCoder's 25.1%. The conceptual contribution: search over candidate Python world-model programs rather than train a neural one.

---

### 0.10 8. World Models for Code Agents

Once an LLM acts as an agent in a non-trivial environment, the world-model question becomes whether the agent simulates the environment's response. Three sub-environments dominate.

#### 0.10.1 8.1 Web agents

Web Agents with World Models (Chae et al., 2024) systematizes the thread. DyMo / World Modeling Improves LM Agents (Guo et al., 2025) adds a next-state prediction head to function-calling agents and reports gains on BFCL-V2, though with a caveat (§17): the WM head reached 90–94% state-prediction accuracy while the underlying policy reached only 72.8% task success, which illustrates that WM-head accuracy and agent accuracy can decouple.

WebDreamer (Gu et al., 2024b) treats the web as a POMDP in which the LLM imagines natural-language state-change descriptions for each candidate click, type, or select. A specialist Dreamer-7B (Qwen2-VL-7B fine-tuned on 3.1M synthesized (initial visual state, action, state-change) tuples from random walks over Common Crawl URLs) provides cheap rollouts. At inference, model-predictive control samples actions, scores simulated trajectories with GPT-4o on a 3-scale rubric, and executes the argmax. On VisualWebArena, Online-Mind2Web, and Mind2Web-Live, this beat the reactive baseline by +34/+42/+24% relative (?+6–11 absolute) while running 4–5× faster than tree search. The bet: when actions are irreversible (forms, purchases), one-step MPC over an LLM-as-WM beats backtracking search.

#### 0.10.2 8.2 OS / computer-use agents

Reinforcement World Model Learning for LLM-based Agents (Yu et al., 2026) and World Models as an Intermediary between Agents and the Real World (Yang, 2026) generalize the lens: a learned WM mediates between LLM and expensive environment.

Dyna-Think (Yu et al., 2025) trains a single Qwen2.5-32B to internalize world-model simulation inside its `<think>` block for OSWorld and WindowsAgentArena. Two stages: DIT (imitation learning on R1 traces cleaned to keep only WM-simulation text) and DDT (Dyna-Q-style joint training over three WM heads — next state, state-diff, critic-prediction — with rejection-sampled policy updates). On OSWorld BoN the 32B model reached 43.1, essentially matching DeepSeek-R1 at 685B with half the tokens. World-model accuracy correlated with task success at r=0.32 across models. Dyna-Think is the corpus's leading instance of policy and learned world model hosted in the same LLM.

#### 0.10.3 8.3 SWE agents

SWE-bench (Jimenez et al., 2023) and SWE-Gym (Pan et al., 2024) defined the eval and training environment respectively. CodeAct (Wang et al., 2024b) made the Python interpreter the unified action space. Reflexion (Shinn et al., 2023) provided the earliest entry with episodic verbal

RL. Nanbeige SWE-World (Sun et al., 2026) trains a learned Docker-free execution surrogate. Understanding by Reconstruction (Zeng et al., 2026) reverses the development process to harvest agentic pretraining traces. SWE-TRACE (Han et al., 2026) provides process-level reward modeling over trajectories. Self-Play SWE-RL (Wei et al., 2025e) introduces adversarial bug-injection/repair self-play. Bootstrapping Coding Agents — The Specification Is the Program (Monperrus, 2026) reframes the SWE task itself as a programmatic spec.

The 2026 wave extended the environment side. Agent World Model ((Research et al., 2026), Snowflake AI Research) generates synthetic environments with executable transition dynamics, treating environment construction itself as a learned capability. CLI-Gym ((Technologies et al., 2026), Huawei) scales CLI-task generation by inverting environment histories rather than scripting tasks by hand. Self-Improving Error Diagnosis ((AI et al., 2026), Amazon Alexa AI) builds verified episodic memory from executable evidence without expert labels. These three target the agent's *training distribution*, not its policy or its WM directly.

The §16 empirical synthesis separates three regimes: execution-grounded open-weight model training (CWM, SWE-RL), execution-grounded agents with learned simulators (Nanbeige SWE-World), and scaffold evolution around closed-model executors (Darwin GM, Huxley GM). Under best-of-k or verifier-reranked protocols, several systems report 60–68% on SWE-bench Verified, but those numbers are not directly comparable to pass@1 model-training results, and the scaffold-evolved systems are not 32B open-weight world-model-trained — they run frontier closed models inside an evolved harness.

#### 0.10.4 8.4 SDLC phase predicts which WM form pays off

The SWE-agent literature aggregates many phases of software development under one benchmark score. Decomposing by phase clarifies which world-model form each phase actually exercises.

- Localization and plan (pre-edit): predominantly retrieval and code-graph reasoning. RepoGraph (Ouyang et al., 2024) and Agentless (Xia et al., 2024) exemplify the phase. WM benefit is small because the agent does not yet simulate execution; retrieval quality dominates.
- Edit generation: where token-space trace-pretrained models help most. CWM, SWE-RL, and the entire §7 lineage land here.
- Debug and test: the phase where forward state prediction would matter most. The agent forms hypotheses about runtime behavior, queries the program to falsify them, and edits the candidate. NExT (Ni et al., 2024), InspectCoder (Wang et al., 2025d), and Agentic Code Reasoning (Ugare and Chandra, 2026) cluster here. This phase is also where probe-style evaluation (§15.2) would be most informative, because the agent's belief state about runtime is observable.
- Verify and deploy: the verifier-grounded line (ATLAS, Re:Form, CLEVER) replaces the WM with a deterministic oracle.

Most SWE-bench gains over the past year accrued to edit-generation; debug-and-test remains the phase where forward-prediction commitments (§3.1 N1–N3) have the clearest mechanism to help and the weakest current evidence.

---

### 0.11 9. RL with Execution as the World Signal

The model-based-RL framing — the world model is what the policy plans over — has produced a clean lineage.

#### 0.11.1 9.1 RLEF (Gehring et al., 2024)

RLEF formulates iterative code synthesis as a POMDP. Actions are full code responses, observations are formatted public-test execution feedback, and rewards come from held-out private tests. Standard PPO with KL regularization and a 3-turn limit. Llama-3.1-70B+RLEF reached 37.5/40.1 pass@1 valid/test on CodeContests at budget 1@3 (vs 25.9/27.5 baseline), matching

AlphaCodium-GPT-4 with 5 samples; at 10@100 it reached 54.5/54.5, surpassing the AlphaCode 41B+clustering baseline. Critically, a random-feedback ablation removed the entire gain, which isolates that the model learns to *use* execution feedback rather than just sample more.

#### 0.11.2 9.2 SWE-RL (Wei et al., 2025d)

SWE-RL applies GRPO to 273k high-quality PR seeds with a rule-based, continuous reward (`difflib.SequenceMatcher` similarity between predicted and oracle patch); no code execution at training time. Llama-3.3-70B fine-tuned this way (Llama3-SWE-RL-70B) hit 41.0% pass@1 on SWE-bench Verified with the Agentless Mini scaffold. The surprising result was OOD transfer: HumanEval+ 76.2?79.9, CRUXEval-O 61.9?75.5, MATH 70.9?73.7, while SFT on the same data *degraded* on these. Continuous reward beat discrete in ablation (34.8 vs 29.0 oracle-repair). The thesis: partial-credit similarity rewards on real PR patches induce reasoning patterns that transfer beyond the training distribution.

#### 0.11.3 9.3 Process Reward Models

ExecVerify (Tang et al., 2026), SWE-PRM (Gandhi et al., 2025), DataPRM (Qiu et al., 2026), and ThinkPRM (Khalifa et al., 2025) form a cluster where a critic over partial trajectories supplies the training signal. PRMs are not forward predictors (§17.4); they complement WMs rather than replace them.

The 2026 wave scaled the verifier idea. Scaling Agentic Verifier ((Team, 2026), Qwen Team) trained a verifier that reasons about candidate-program behaviors and discovers counterexamples rather than just scoring tokens. V1 ((Berkeley et al., 2026), Berkeley + Together AI + Mila) unifies generation and self-verification so the same model produces candidates and pairwise verification judgments. Both move PRMs closer to forward-prediction territory: the verifier now reasons about what the program *would do*, not only about whether a step *looks right*.

---

### 0.12 10. Planning and Search with Code World Models

#### 0.12.1 10.1 RAP (Hao et al., 2023)

RAP frames reasoning as MCTS in a self-consistent MDP where the same frozen LLM serves as both policy and transition model. A state is a textual configuration (blocks layout, intermediate variables, current fact), an action is a step proposed by the LLM, and the transition is obtained by re-prompting. Rewards combine action likelihood, state confidence (majority voting), self-evaluation, and task heuristics. On Blocksworld 4-step, RAP@10 reached 0.86 with LLaMA-33B, surpassing GPT-4+CoT's 0.63 by 33% relative. The conceptual template — repurpose the LLM as both policy and transition model under MCTS — is what every later LLM-as-WM paper extends.

Tree of Thoughts (Yao et al., 2023), AlphaZero-like Tree Search for LLM Decoding (Feng et al., 2023), Tree Search for LM Agents (Koh et al., 2024), and Mastering Board Games by External/Internal Planning with LMs (Schultz et al., 2024) develop the search frame. The last gives the most direct contemporary recipe for learned tree-search with LLM-as-WM, straightforwardly transferable to code.

#### 0.12.2 10.2 Execution-conditioned generation

Execution Guided Line-by-Line Code Generation (Lavon et al., 2025) uses classifier-free guidance to condition next-token prediction on candidate-runtime outcomes. Jupiter (Li et al., 2025a) formulates notebook state as MCTS nodes. REPL-Plan (Liu et al., 2024) reuses a REPL state pool across tasks. The substrate is well-developed for short-horizon code-gen, less so for long-horizon multi-file SWE.

---

## 0.13 11. JEPA, Dreamer, and the Latent-Action Gap

LeCun's Joint Embedding Predictive Architecture (I-JEPA, (Assran et al., 2023)) predicts in embedding space rather than pixel space. The Dreamer family — Hafner et al.'s DreamerV1 (Hafner et al., 2019), DreamerV2 (Hafner et al., 2020), and DreamerV3 (Hafner et al., 2023), built around the Recurrent State-Space Model — has near-zero direct application to code. Two papers occupy the gap.

### 0.13.1 11.1 LLM-JEPA (Huang et al., 2025)

LLM-JEPA adds a joint-embedding predictive objective to standard NTP training, using (text, code) as the two JEPA views with the LLM's last-layer last-token hidden state as encoder and a tied-weights `[PRED]` token as predictor. The loss is `L_NTP(text) + λ · d(Pred(Enc(Text)), Enc(Code))` with cosine distance. On Llama-3.2-1B fine-tuned on NL-RX-SYNTH the gain was 57.3 → 71.5 (+14.2 absolute); on Spider, GSM8K, and HellaSwag the wins were smaller. The top-100 singular values of `Enc(Text) − Enc(Code)` collapsed by orders of magnitude, which indicates a low-rank text→code mapping. Whether this counts as JEPA in the LeCun sense or as a regularizer on top of NTP remains open and is discussed in §17.

### 0.13.2 11.2 CoLA (Jia et al., 2025)

CoLA replaces the 128k-token action space of an LLM with a small learned latent-action codebook. Three modules: a VQ-VAE-style inverse-dynamics model that infers latent action $a_t$ from $(x_{1:t}, x_{t+1})$; a language world model that inserts the chosen latent action into the LLM embedding stream and decodes the next token; and a policy $\pi(a_t \mid x_{1:t})$ behavior-cloned from inverse-dynamics labels then RL-tuned. Action-level MCTS over the learned codebook (with a Double-DQN Q-function) reached Math-500 68.2 vs 63.0 baseline MCTS-Q. CoLA is the corpus's most direct Dreamer-for-LLMs instance: the action space is genuinely compressed, and rollout and search operate in that compressed space.

### 0.13.3 11.3 The gap

Despite CWM and dozens of LLM-as-world-model papers, no public Dreamer/RSSM-style latent-imagination world model has been trained for SWE agents. CWM rolls out in token space. CoLA is the closest concrete instance. UniZero (Pu et al., 2024) generalizes MuZero with transformers but is rarely instantiated on code. Genie (Bruce et al., 2024) gives the vision-side template. JEPA for RL (Kenneweg et al., 2025) extends the energy-based objective to RL.

Whether the gap matters remains open and is developed critically in §17. The vision-domain pressure that motivated Dreamer's RSSM design (pixel-space rollout cost) does not exist for code, where state is small and the simulator is available. The argument *for* latent imagination rests on inference-time speed and the action-space compression CoLA demonstrates, not on rollout cost per se.

---

## 0.14 12. Specialized Domains

Diffusion code models. DiffuCoder (Gong et al., 2025), Dream-Coder 7B (Xie et al., 2025). Iterative denoising accommodates plan-then-refine generation.

Decompilation and cross-language. SK2Decompile (Tan et al., 2025), SALT4Decompile (Wang et al., 2025c). Translation as semantic-simulation task. EquiBench (Wei et al., 2025a) supplies the equivalence eval.

Hardware / RTL. VeriRL (Teng et al., 2025), ChipSeek (Chen et al., 2025b), and VeriCoder (Wei et al., 2025c) form a cluster where the simulator is the world model. Hardware suits this approach because simulators are precise, fast, and deterministic, closer to Atari than to Python.

ARC and abstract synthesis. Executable World Models for ARC-AGI-3 (Rodionov, 2026) instantiates literal-WM-per-task: synthesize a Python world model verified against observations. SOAR

(Pourcel et al., 2025) evolves programs over ARC. Darwin and Huxley Godel Machines ((Zhang et al., 2025a), (Wang et al., 2025b)) close the self-improvement loop.

---

## 0.15 13. Reasoning, Process Rewards, Memory

Long-CoT reasoning for code. o1-Coder (Zhang et al., 2024c) replicates o1 with MCTS+RL. R1-Code-Interpreter (Chen et al., 2025a) supplies the open SFT+RL recipe across 144 tasks. Scaling Test-Time Compute to Achieve IOI Gold Medal (Samadi et al., 2025) shows open-weight gpt-oss-120b matching closed reasoning models via inference-time scaling.

Long-CoT reasoning amounts to mental execution: the chain-of-thought simulates the world model the network never explicitly trained. CWM-style explicit world modeling and R1-style reasoning are partial substitutes. Whether they compose multiplicatively remains open.

Memory. Episodic Memory is the Missing Piece for Long-Term LLM Agents (Pink et al., 2025) frames the gap. Memory as Action (Zhang et al., 2025b) treats memory operations as RL-learnable actions. RepoGraph (Ouyang et al., 2024) provides a durable repo-level dependency graph.

---

## 0.16 14. Verification, Probing, Safety

### 0.16.1 14.1 Formal verification

The verifier-grounded lineage is the only research direction in the corpus that does not rely on LLM self-report for correctness; the verifier provides ground truth.

ATLAS (Baksys et al., 2025) provides the most thorough end-to-end pipeline in the corpus for scaling verified-code data. From TACO-verified (12.8k LeetCode-style problems with Python references and tests), ATLAS produced 2,751 verified Dafny programs decomposed into 19,385 training examples across six tasks (NL-to-Code, NL-to-Spec, Spec-to-Code, Spec-Repair, Impl-Repair, Proof-Infilling). Spec quality was filtered by three SMT-discharged lemma types: soundness, completeness-contradiction, and completeness-perturbation. Qwen-2.5-Coder-7B fine-tuned this way reached DafnyBench Pass@1 of 55.8% (from 32.4%) and DafnySynthesis Pass@5 of 65.8% (from 15.8%, surpassing GPT-4's 53.4). ATLAS does for verified Dafny what auto-formalization did for Lean theorems.

The cluster also includes Re:Form ((Yan et al., 2025), Dafny+RL), CLEVER ((Thakur et al., 2025), Lean), VeriStruct ((Sun et al., 2025), Verus), AutoRocq ((Tu et al., 2025), Rocq), and Semantic Equivalence Self-Play with Formal Verification ((Barone and Nok, 2026), Liquid Haskell).

CLEVER's □1/161 end-to-end Lean result is the most sobering number in the corpus: frontier models, with Lean type-checker access for self-verification, still fail on >99% of HumanEval-derived problems requiring joint spec and implementation verification. Understanding Formal Reasoning Failures in LLMs as Abstract Interpreters (Mitchell et al., 2025) supplies the diagnostic: when asked to reason in the style of formal abstract interpretation over 22 SV-COMP programs, all frontier reasoning models made systematic errors in widening, fixpoint termination, and join operations.

### 0.16.2 14.2 Symbolic execution and LLMs

AutoBug (Li et al., 2025c), SESpec (Yang et al., 2025), LLM-Sym (Wang et al., 2024a), and Loop Invariant Generation via Reasoning LLMs + SMT (Bharti et al., 2025) combine LLMs with concrete or symbolic engines. The unifying pattern: the LLM hypothesizes the world model; symbolic execution verifies or extends it.

### 0.16.3 14.3 Probing and mechanistic interpretability

Mechanistic Interpretability of Code Correctness via SAEs (Tahimic and Cheng, 2025) and On LLMs' Internal Representation of Code Correctness (Ribeiro et al., 2025) ask what code LLMs

actually represent. Findings: partial, brittle internal execution representations, which supports explicit trace pretraining.

#### 0.16.4 14.4 Repair and debugging as world-model probing

Self-Debug (Chen et al., 2023), InspectCoder (Wang et al., 2025d), Agent That Debugs — Dynamic State-Guided Vulnerability Repair (Liu et al., 2025), and Agentic Code Reasoning ((Ugare and Chandra, 2026), semi-formal execution-path reasoning without running code) share a pattern: maintain a belief over program state, query the runtime to update the belief, act on the posterior — Bayesian world-modeling in everything but name.

#### 0.16.5 14.5 Safety and malicious code

The Double Life of Code World Models (Sahoo, 2025) repurposes CWM-style trace predictions for malicious-behavior detection. CodeBreaker (Yan et al., 2024) provides the offensive analogue. Concolic Execution + LLM for Zero-Day Malware Detection (Edwards and Eslamimehr, 2026) pairs path-prioritization with concrete execution.

---

### 0.17 15. Three Regimes of Evaluation for Code World Models

Benchmarks for code world models fall into three regimes that differ in what they actually measure. The distinction matters because most current evaluation conflates them, and the missing third regime is what would allow direct measurement of WM quality.

#### 0.17.1 15.1 Endpoint evaluations (what the system produces)

Endpoint benchmarks score the system's final output against a held-out test. SWE-bench (Jimenez et al., 2023), HumanEval, MBPP, LiveCodeBench (Jain et al., 2024), and PyBench (Zhang et al., 2024a) all fit here. They measure end-to-end task success without isolating any internal capability. A system can score well on endpoint benchmarks by exploiting test-distribution shortcuts, by retrieving similar solutions, or by genuinely simulating program semantics; the score alone cannot distinguish these mechanisms. Current SWE-bench leaders all use endpoint evaluation, which is why §16.1 stratifies by protocol rather than ranking by score.

#### 0.17.2 15.2 Process probes (what the system internally simulates)

Process probes hold the task fixed but require the system to surface intermediate state — execution traces, variable bindings, branch coverage. CRUXEval (Gu et al., 2024a), REval (Chen et al., 2024), CRUXEval-X (Xu et al., 2024), TraceEval (Li et al., 2026), and PLSemanticsBench (Thimmaiah et al., 2025) all fit here. EquiBench (Wei et al., 2025a) and CodeARC (Wei et al., 2025b) extend the regime to semantic equivalence. Process probes are the closest existing measurement of WM quality: they ask whether the model can simulate execution rather than just produce a final answer.

Demystifying Errors in LLM Reasoning Traces (Abdollahi et al., 2025) exposed the limit of this regime. When DeepSeek-R1, o4-mini, Gemini 2.5 Flash, and Claude 4 were prompted to simulate execution and explain their reasoning, the produced traces contained errors clustered into nine categories: computation, indexing, control flow, skipped statements, native-API misvaluation, hallucination, input misreads, and others. Models with 85–98% accuracy on final-output prediction produced traces with systematic intermediate errors. Outcome accuracy decouples from process fidelity, which is the signature of a system that learned to predict outputs without faithfully simulating dynamics.

ProgramBench (Yang et al., 2026a) sits between regimes. Meta FAIR and Princeton (the SWE-bench team) released it in May 2026; it asks whether language models can rebuild whole programs from scratch given only natural-language specifications, evaluated by behavioral equivalence. The behavioral-equivalence rubric is closer to a process probe than an endpoint score, because two programs can pass the same unit tests by accident but rebuilding implies semantic reconstruction.

### 0.17.3 15.3 Counterfactual probes (missing)

The third regime would hold the policy fixed and vary the world model. Concretely: take a fixed coding policy, swap in different trace-pretraining recipes or different forward-prediction heads, and measure the delta on both process and endpoint metrics. No current benchmark supports this protocol. CWM, TRACED, NExT, and SemCoder each report endpoint and process gains, but none isolates the WM contribution from the policy contribution, because the policy and the WM live in the same weights.

The counterfactual regime is what §17.7 names as the structural gap. Closing it requires either explicit forward-prediction modules (§3.1, N1–N3) that can be ablated independently of the policy, or matched-harness re-runs across the trace-pretraining lineage. A concrete proposal: fix the Qwen-2.5-Coder-32B base, vary the trace-pretraining mixture across the recipes used by TRACED, NExT, SemCoder, and CWM, and measure CRUXEval and SWE-bench separately. The resulting two-dimensional table would, for the first time, allow a regression of process fidelity onto endpoint success.

---

## 0.18 16. Empirical Landscape

### 0.18.1 16.1 SWE-bench scoreboard

Protocols mix in SWE-bench reporting and are non-comparable across the obvious axes. The table below splits into three protocol classes; readers should compare within, not across, classes. Training / Evaluation regime: *Frozen-model scaffold* = no model training (agent loop around a frozen closed model); *EG-LLM SFT* = supervised fine-tune on agent trajectories; *EG-LLM + RL* = RL with execution rewards; *EG-agent + learned simulator* = SFT+RL plus a learned execution surrogate; *EG-LLM + trace mid-train + RL* = the CWM lineage; *Scaffold-evolved* = recursive scaffold/agent self-improvement around a frozen closed model. WM-Arch is reserved for systems with explicit transition/rollout machinery (N1/N2/N3 from §3); no row in this table qualifies.

Table 16.1a — Single-sample resolution (pass@1 on Verified unless noted)

| System | Base model | Bench | Pass@1 | Regime | Source |
|---|---|---|---|---|---|
| SWE-agent | GPT-4 Turbo | full | 12.5% | Frozen-model scaffold | (Yang et al., 2024) |
| AutoCodeRover | GPT-4 | Lite | 19.0% | Frozen-model scaffold | (Zhang et al., 2024b) |
| Agentless | GPT-4o | Lite | 32.0% | Frozen-model scaffold | (Xia et al., 2024) |
| SWE-Gym | Qwen-2.5-Coder-32B | Verified | 20.6% | EG-LLM SFT | (Pan et al., 2024) |
| Agent-RLVR (no RM) | Qwen-2.5-72B | Verified | 22.4% | EG-LLM + RL | (Da et al., 2025) |
| Long-Context Multi-Turn RL | Qwen-2.5-72B | Verified | 39.0% | EG-LLM + RL | (Golubev et al., 2025) |
| SWE-RL (Llama3-SWE-RL-70B) | Llama-3.3-70B | Verified | 41.0% | EG-LLM + RL | (Wei et al., 2025d) |
| Nanbeige SWE-World (RL) | Qwen-2.5-Coder-32B | Verified | 55.0% | EG-agent + learned simulator | (Sun et al., 2026) |

| System | Base model | Bench | Pass@1 | Regime | Source |
|---|---|---|---|---|---|
| CWM | CWM-32B | Verified | *not directly reported* † | EG-LLM + trace mid-train + RL | (FAIR CodeGen Team et al., 2025) |

† The CWM paper does not report a pure pass@1 number under the standard protocol. The headline 65.8% in Table 16.1b is best@16 with verifier reranking. Inferred pass@1 from the paper's reported figures is approximately 53–55%, but this is an estimate from the survey authors, not a directly reported number, and is not placed in the main row.

Table 16.1b — Best-of-k with verifier reranking / TTS

| System | Base | Bench | Score | Sample budget | Source |
|---|---|---|---|---|---|
| Agent-RLVR + RM rerank | Qwen-2.5-72B | Verified | 27.8% | rerank over multi-sample | (Da et al., 2025) |
| Nanbeige SWE-World (TTS@8) | Qwen-2.5-Coder-32B | Verified | 68.2% | best@8 | (Sun et al., 2026) |
| CWM (TTS) | CWM-32B | Verified | 65.8% | best@16 over 40 reranked | (FAIR CodeGen Team et al., 2025) |

Table 16.1c — Scaffold-evolution and self-play around closed-model executors

| System | Backbone executor | Bench | Score (start → end) | Note | Source |
|---|---|---|---|---|---|
| Darwin Godel Machine | Claude-3.5-Sonnet | Verified | 20% → 50% | Scaffold evolves over 80 iterations | (Zhang et al., 2025a) |
| Huxley Godel Machine | GPT-5-mini | Verified (500) | — → 61.4% | Scaffold optimized for Verified-60 | (Wang et al., 2025b) |
| SICA | Claude-3.5-Sonnet + o3-mini | Verified (subset) | 17% → 53% | Scaffold evolves; not a model-training method | (Robeyns et al., 2025) |

Citations that quote CWM at 65.8% on SWE-bench Verified without disclosing the best@16/TTS protocol misreport the result rather than reporting direct pass@1 numbers.

What can and cannot be compared. Within Table 16.1a the comparable axis is pass@1. SWE-RL's 41.0%, Long-Context-MT-RL's 39.0%, and Nanbeige SWE-World's 55.0% are roughly comparable as open-weight WM-trained pass@1 on Verified. Table 16.1c sits in a different category: these are scaffold-evolution methods over closed models, and treating them as evidence for world-model training conflates scaffold search with model improvement. Reading them inside Table 16.1a alongside model-trained pass@1 numbers is apples-to-oranges.

Two empirical claims, separately defensible. (i) On *pass@1*, open-weight WM-trained 32B systems (Nanbeige 55.0%, Long-Context-MT-RL 39.0%, SWE-RL 41.0%) outperform frozen open-weight baselines (Qwen-2.5-Coder-32B raw 6.2%) by 30–50 absolute points. This gain is the strongest case for training on agent trajectories with execution feedback. (ii) On *best@k with*

*verifier reranking* (Table 16.1b), the same models reach 65–68%, but the additional 13–15 points are attributable to TTS reranking infrastructure, not to the WM training. Conflating (i) and (ii) by quoting CWM's 65.8% alongside SWE-RL's 41.0% as both world-model-trained pass@1 SOTA is exactly the misreporting the protocol split is designed to prevent.

### 0.18.2 16.2 Trace pretraining gains on execution-reasoning

| Paper | Backbone | Baseline | After trace pretrain/FT | Delta | Benchmark |
|---|---|---|---|---|---|
| TRACED | UnixCoder | — | +12.4% rel branch-coverage; +25.2% rel variable-value | rel | CodeNet exec |
| NExT | PaLM 2-L | 23.2 | 49.3 | +26.1 abs | MBPP-R |
| NExT | PaLM 2-L | 32.2 | 42.5 | +10.3 abs | HumanEvalFix-Plus |
| SemCoder (1.3B) | DS-Coder 1.3B | base | 63.6 / 63.9 | +23 abs | CRUXEval-I / O |
| What I cannot execute | Llama-3.1-8B | 37.8% | ~80% | +42 abs | CRUXEval-O |
| Do Code Semantics Help? | DSCoder, Llama-3, Gemma-2 | various | □ a few abs; some regressions | mixed | comprehensive |

For under-trained □8B open-weights, trace pretraining delivered +15 to +42 absolute on CRUXEval-O. The Do Code Semantics Help? ablation (Wang et al., 2025a) supplied the disconfirming evidence: across multiple backbones and five trace representations, no single representation consistently outperformed others, and several downstream tasks regressed under trace augmentation. The gain shrinks rapidly with base-model quality. Trace pretraining functions as a remedial intervention for weak code models. Whether frontier models still benefit remains unsettled.

### 0.18.3 16.3 Web/OS agents from WMs

| Paper | Benchmark | Base | With WM | Delta | Mechanism |
|---|---|---|---|---|---|
| WebDreamer (GPT-4o WM) | VisualWebArena | 17.6% | 23.6% | +34.1% rel (□+6 abs) | LLM-as-WM + MPC |
| WebDreamer | Online-Mind2Web | 26.0% | 37.0% | +42.3% rel (□+11 abs) | same |
| Dreamer-7B (trained WM) | VisualWebArena | base | +4.7 abs | — | trained WM |
| WMA (Chae et al., 2024) | WebArena | base | action-selection 52□70% | — | trained transition WM |
| Dyna-Think DDT (32B) | OSWorld BoN | RFT~28% | 43.1% | □+15 abs | Dyna-Q + WM head |
| Dyna-Think DDT | WindowsAgentArena | 28.4% | 34.9% | +6.5 abs | same |

WM gains on web/OS are real but quantitatively small (□+5–10 absolute task success rate on most benchmarks), and partly confounded with the extra synthetic data the WM generates.

DyMo's 90%+ state-prediction accuracy versus 72.8% task success rate exemplifies the decoupling: WM heads can be accurate without the agent being accurate.

### 0.18.4 16.4 Formal verification vs LLM-only

| System | Language | Benchmark | Baseline | With system | Source |
|---|---|---|---|---|---|
| ATLAS | Dafny | DafnyBench Pass@1 | 32.4% | 55.8% | (Baksys et al., 2025) |
| ATLAS | Dafny | DafnySynthesis Pass@5 | 15.8% | 65.8% (>GPT-4 53.4) | (Baksys et al., 2025) |
| CLEVER | Lean 4 | 161 problems end-to-end | best frontier | ?1/161 | (Thakur et al., 2025) |
| VeriStruct | Verus / Rust | 11 modules | — | 99.2% (128/129 fns) | (Sun et al., 2025) |
| AutoRocq | Rocq | math + verif lemmas | 5 baselines | 48.0% math / 30.9% verif | (Tu et al., 2025) |
| Semantic Equiv Self-Play | Liquid Haskell | EquiBench | base | +13.3 pp | (Barone and Nok, 2026) |

Verified codegen shows the steepest training-data sensitivity in the survey: small synthetic datasets (2.7K verified Dafny programs in ATLAS) produce +25–50 absolute gains because LLM-only baselines start near zero. CLEVER's ?1/161 shows that without explicit data or scaffolding, frontier models cannot reliably produce verified code. VeriStruct shows that on curated targets, near-perfect performance is reachable.

### 0.18.5 16.5 Reasoning-model competitive programming

| Model | Codeforces | IOI / ICPC |
|---|---|---|
| gpt-4o | 808 (11th pct) | — |
| o1-preview | 1258 (62nd pct) | — |
| o1 | 1673 (89th pct) | — |
| o1-ioi | 1807 (~93rd pct) | IOI 2024 49th pct live |
| o3 | elite-human-class | IOI 2024 gold |
| gpt-oss-120b + GenCluster | — | IOI 2025 gold (open-weight) |
| Gemini 2.5 Pro Exp | — | ICPC-Eval Pass@1 22.0% |
| DeepSeek-R1 | — | ICPC-Eval Pass@1 14.4% |
| Claude 3.7 Sonnet | — | ICPC-Eval Pass@1 11.8% |
| GPT-4o | — | ICPC-Eval Pass@1 5.9% |

Codeforces ratings rose by 999 points in 14 months on the o-series. The same line shows that frontier reasoning models gain almost everything from RL scaling, not from domain-specialized scaffolds, which complicates the case that trace-style world models do causal work for frontier systems.

---

## 0.19 17. Critical Perspectives

This section examines where consensus is fragile and where evidence does not yet support common framings. Seven theses follow.

### 0.19.1 17.1 Most current code WMs make data choices, not architectural choices

The empirical center of the field has converged on a particular design: a standard decoder-only LLM trained on an enriched corpus that includes execution traces or agent trajectories. CWM (FAIR CodeGen Team et al., 2025) is the largest instance — a 32B Llama-class Transformer with GQA, sliding-window blocks, RoPE, mid-trained on 5T tokens of Python observation-action traces plus 3M ForagerAgent SWE trajectories. There is no separate dynamics head, no inverse model, no recurrent latent. TRACED, NExT, SemCoder, and LLM-JEPA share the pattern: data-side innovation, architecturally standard.

This is a substantive empirical finding, not a complaint. It says that current code-WM gains are achievable without architectural commitment to forward-prediction machinery. Whether deeper commitments (the N1/N2/N3 forms in §3.1) would yield gains beyond what enriched data already provides is the central open question. CoLA (Jia et al., 2025) is the only system in the corpus that goes further architecturally; its gains are positive but modest. The architectural-commitment question is the one §17.3 and §18 develop.

### 0.19.2 17.2 Trace pretraining has a causal-isolation problem the surface numbers obscure

Do Code Semantics Help? (Wang et al., 2025a) supplies the most damaging paper for the prevailing optimism. It ran a comprehensive ablation across DeepSeek-Coder, LLaMA-3, and Gemma-2 with five representations (Scratchpad, NExT, CodeExecutor, Concise, SemCoder) on program repair, code synthesis, BigCodeBench, LiveCodeBench, and CRUXEval. Its headline: integrating trace-based semantic information into SFT does not significantly enhance code-generation ability. In 7 of 9 synthesis settings the no-trace baseline won or tied. At inference, in 36 of 56 test-scaling configurations, trace prompts hurt.

What I cannot execute, I do not understand (Armengol-Estapé et al., 2025) is gentler: Execution Tuning reached ~80% CRUXEval-O. But the same paper's downstream evaluations on HumanEval, MBPP, and GSM8K showed negligible gains from trace data in the SFT mix.

The strongest counterexample to barely-transfers is NExT (Ni et al., 2024), which pushed PaLM 2-L on Mbpp-R from 23.2% to 40.8% with traces removed at test time: a +17.6 absolute gain on the no-trace-at-inference setting, exactly the transfer pattern the skeptical reading says should not happen. The narrower version of the thesis is therefore: trace pretraining transfers to program-repair tasks where the model needs to reason about a buggy program's behavior (where NExT and TraceFixer-style work shines), and barely transfers to fresh code synthesis on benchmarks like HumanEval and MBPP that sit closer to the model's prior. The Do Code Semantics Help? disconfirmation is on synthesis; NExT's transfer is on repair. Both can hold.

The interpretation that survives: trace pretraining helps execution prediction (the thing trained on), transfers to runtime-reasoning-heavy downstream tasks like repair, and barely transfers to fresh code synthesis. This matches the pattern expected if dense execution supervision teaches a runtime-tracking skill rather than a generative model of program intent.

CWM's 65.8% on SWE-bench looks like a counterexample, but the Meta team reports it after trace mid-training *and* 3M ForagerAgent SWE trajectories *and* multi-task RL with verifiable rewards — three interventions stacked. The world-model component is not causally isolated from the SWE-trajectory and RL components. Without an ablation that removes trace mid-training while holding ForagerAgent and RL fixed, the headline remains unfalsifiable. The field has agreed to call CWM a world-model success because the name appears on the model card, not because the experimental design demonstrates it.

### 0.19.3 17.3 The Dreamer-for-code gap may be a non-problem

The conventional framing treats the absence of latent-imagination world models for SWE as the field's largest architectural gap. The empirical record argues the opposite. CWM in token space reaches 65.8% SWE-bench. CoLA produces respectable but not field-shifting results, and even there the WM is fine-tuned on top of a standard LLM rather than replacing it.

Vision world models needed latent rollouts because pixel-space rollouts were too expensive: a frame is ~10^6 dimensions, dynamics are partially observed. Program execution is the opposite:

a Python frame is small, dynamics are observable, and the simulator (CPython) is available for free at training time. The pressure that drove Dreamer's RSSM design does not exist for code.

Debugging Code World Models (Rahmani, 2026) showed that CWM's long-horizon failures are dominated by action hallucination, not state-propagation error. Under teacher forcing CWM tracked state correctly for 128 steps. A latent rollout would compress states but not fix the action policy, which is the actual bottleneck.

The counterargument is fair: latent rollouts permit faster planning at inference, and for multi-agent or population-scale search the speedup is asymptotically meaningful. The survey should retire the *single largest architectural gap* framing and replace it with *an open question whose payoff is not yet demonstrated*.

#### 0.19.4 17.4 PRMs and forward WMs solve complementary problems

Process reward models — ExecVerify (Tang et al., 2026), SWE-PRM (Gandhi et al., 2025), ThinkPRM (Khalifa et al., 2025), FunPRM (Zhang et al., 2026), DataPRM (Qiu et al., 2026) — score partial trajectories. Forward world models predict `(state, action) → next_state`. In classical model-based RL these play different roles: Dreamer has both a world model (RSSM) and a critic (value function), and the two interact during planning.

In code, the same complementarity holds. A PRM trained on execution traces can score whether a candidate next step looks promising; a forward WM can simulate what that step would actually produce. Current code-WM systems mostly do one or the other, and the design space for combining them is largely unexplored. The cleanest path forward would integrate an execution-grounded PRM as the critic, with a forward predictor (token-space, latent, or synthesized) supplying the candidate rollouts the critic scores.

#### 0.19.5 17.5 General Agents Contain World Models is much weaker than its title suggests

Richens et al. (Richens et al., 2025) prove that any goal-conditioned policy satisfying a regret bound $\delta$ for sufficiently deep composite goals (depth $n \square 1$) must encode an extractable approximation of the transition function with bounded error. The result is genuine and elegant.

But read the assumptions: fully observed environment, finite communicating stationary controlled MDP, goal-conditioned policy satisfying a regret bound for a specific class of LTL composite goals of depth n. Theorem 2 of the same paper explicitly shows that for myopic agents (depth-1 goals), no world model is needed. Real SWE agents are myopic-ish over short turns and approximately competent over longer ones; their environments are partially observed (rarely full filesystem state); they violate stationarity (the repository changes under their actions); and their regret bound for arbitrary composite goals is unknown and almost certainly not satisfied. The authors caveat this in §6 (Limitations) of their paper.

The theorem provides an existence proof for an idealized agent class. It is not an empirical statement that SWE coding agents have learned world models, and it provides no guidance about the fidelity of any world model they may have learned.

#### 0.19.6 17.6 The verifier-grounded lineage leads under a narrow scope

ATLAS, Re:Form, CLEVER, VeriStruct, AutoRocq, and the Liquid Haskell self-play paper (Barone and Nok, 2026) share a property no LLM-only system possesses: code whose correctness is machine-checked against a formal specification. Compare to the SWE-bench paradigm, where correctness means hidden unit tests pass — a weaker guarantee, because unit tests cover specific inputs and a system can pass them while being wrong on adjacent inputs.

The abstract-interpreter paper (Mitchell et al., 2025) provides the diagnostic: when frontier reasoning LLMs were asked to reason in the style of formal abstract interpretation over 22 SV-COMP programs, they made systematic errors in widening, fixpoint termination, control-flow propagation, and meet/join operations. They generated unsound invariants on programs as small as `count_by_2.c`. If LLMs cannot reliably perform interval-domain abstract interpretation on toy C programs, claims that they have learned faithful internal world models of program semantics demand a lot of inferential work.

The verifier-grounded line is the only research direction that does not rely on LLM self-report for correctness. Scoped carefully, it leads on synthesis-from-spec (ATLAS reached 65.8% Pass@5 on DafnySynthesis at 7B, surpassing GPT-4) and on near-perfect verified completion of curated targets (VeriStruct 99.2% on 11 Verus modules). It does not lead on end-to-end verified codegen from natural language: CLEVER reported ?1/161 on Lean problems requiring joint spec and implementation verification. The future of correct code is plausibly hybrid — neural proposal, symbolic verification, with the verifier providing ground truth that learned world models cannot — but the hybrid is not yet a finished pillar. The verifier line is cataloged in §14.1 alongside symbolic execution and abstract-interpretation probing; readers who accept this thesis should weight that subsection more heavily than its sequence position suggests.

### 0.19.7 17.7 The evaluation gap is structural

Across CRUXEval, REval, CRUXEval-X, PLSemanticsBench, TraceEval, and EquiBench, no benchmark holds policy fixed and varies world-model quality. Every measurement of world-modeling capability is mediated through downstream task performance, so the field cannot distinguish *the model has internalized program semantics* from *the model is exploiting trace-token shortcuts in the test distribution*.

Demystifying Errors in LLM Reasoning Traces (Abdollahi et al., 2025) provides the diagnostic: even DeepSeek-R1, o4-mini, Gemini 2.5 Flash, and Claude 4, when asked to simulate execution, produced traces with errors in nine systematic categories. Models with 85–98% final-answer accuracy on output prediction produced traces with systematic errors throughout. High outcome accuracy, low process fidelity: the signature of a system that predicts outcomes without faithfully simulating dynamics.

Self-repair literature exhibits the same pathology. Olausson et al. (Olausson et al., 2023) showed that GPT-4 self-repair on APPS and HumanEval, normalized by compute, often performs worse than i.i.d. resampling. The bottleneck is the model's feedback quality, not its repair capability. Human-written feedback boosts repair success by 1.58×. The model can generate code, can sometimes recognize bugs, but cannot reliably simulate why its code is wrong, which is exactly what a faithful world model would let it do. The empirical bound on LLM self-repair is, in effect, an empirical bound on the fidelity of the implicit world model the LLM is running. Calling that world model internal is fine; calling it good is not.

Until benchmarks measure process fidelity independently of outcome, *is this system actually building a world model?* remains scientifically undecidable. §15.3 names the missing regime — counterfactual probes that hold policy fixed and vary WM quality — and proposes a concrete protocol: fix the Qwen-2.5-Coder-32B base, vary the trace-pretraining mixture across the TRACED, NExT, SemCoder, and CWM recipes, and measure CRUXEval and SWE-bench separately. The resulting matrix would, for the first time, allow regressing process fidelity onto endpoint success and isolating which trace recipes transfer.

---

## 0.20 18. Open Problems

The critical perspectives of §17 reshape the conventional open-problems list. Six problems where the literature is thinnest and the upside is largest:

1. Causal isolation of trace-pretraining contributions. Every claim of the form *this WM-trained model achieves X* should be paired with an ablation removing the WM component while holding training data and RL fixed. CWM in particular needs this. Without it, the headline numbers underdetermine whether the WM did the work.

2. World-model fidelity as a first-class metric. §15's eval gap is concrete: build a benchmark where holding policy fixed and varying WM quality causes measurable variation in planning quality, independent of downstream task. This benchmark would clarify the field more than any single new model.

3. Hybrid neural-symbolic systems. §17.6 argues the verifier-grounded line leads on its scope. The natural integration is neural proposal, symbolic verification, with the verifier providing

gradient-free correctness signal and the neural component providing proposals at scale. Differentiable surrogates of symbolic verifiers (Lean, Dafny, Rocq) that pass verifier-style gradients during training remain open.

4. Multi-modal WMs for coding. GUI agents need pixel-level WMs (Neural Computers, (Zhuge et al., 2026), provides a first attempt). Tying pixel WMs to code-state WMs through a shared latent remains essentially unsolved.

5. Long-horizon credit assignment with execution-grounded rewards. PRMs (§9.3) are early, and §17.4 argues they should be conceptually separated from world models. The right structure for rewarding an agent across hundreds of execution-grounded steps remains open.

6. World models of the developer, not just the program. All current WMs model the machine. Few model the developer intent with comparable fidelity. ATLAS and Re:Form gesture in this direction by treating the spec as the WM. A full developer-intent WM would close the agentic loop.

Dreamer-for-SWE-agents is not listed as the field's largest gap. §17.3 argues the pressure motivating that direction in vision does not transfer to code. It remains an interesting research question, not the highest-leverage one.

Three under-explored representations. The §5.3 taxonomy table flags three classes of representation, mature in vision-WM, that have no code-WM exemplar:

- *Global Latent Vector (Dreamer-style RSSM)* — discussed and contested above. The Hafner et al. DreamerV1–V3 line proves the design space exists; whether it pays off for code is the question §17.3 leaves open.
- *Spatial / Structural Grid* — the analog of OccWorld / BEV for code would be a learned predictive grid over AST nodes, call-graph edges, or CFG states. RepoGraph (Ouyang et al., 2024) shows the static version is useful as agent state; the predictive version remains unexplored.
- *Decomposed Object / Slot (object-centric WMs)* — the analog for code would model variables, scopes, or classes as discrete persistent slots whose state propagates independently. No paper in the corpus instantiates this, despite obvious mappings (each variable is an object, each frame is a scene).

The object-centric and structural-grid gaps look more genuine than the Dreamer one, because they exploit structure that code already has (objects = variables, grids = AST/CFG) rather than borrowing pressure from a domain (vision) where the structural assumptions differ.

---

## 0.21 19. Conclusion

Across the literature surveyed here, a single trajectory is visible: from neural execution (modeling the machine), through trace pretraining (modeling execution implicitly), to CWM and its descendants (modeling execution explicitly with a named artifact), to agentic SWE and RL (modeling the environment), to JEPA and latent-action models (modeling in compressed space), and on toward formal verification, probing, and safety (modeling reliably). What was a scattered set of insights in 2014 has by 2026 cohered into a recognizable research program with a recognizable artifact: the code world model.

The remaining work splits into two halves. The first is empirical: close the eval gap, isolate the causal contribution of WM-training, build hybrid neural-symbolic systems whose correctness is verifier-checkable rather than test-checkable. The second is rhetorical: hold the term *world model* to a strict definition so the literature can distinguish architectural commitments from training-data choices, and resist the temptation to oversell extractability theorems and latent-imagination analogies whose premises do not transfer to code.

The opportunity is large precisely because the framework is now clear enough to identify what is missing. The work to do is the work this survey has tried to make visible.

---

## References


Abdollahi, M., Tasnia, K. R., Saha, S. K., Yang, J., Wang, S., and Hemmati, H. (2025). Demystifying errors in llm reasoning traces: An empirical study of code execution simulation.

AI, A. A. et al. (2026). Towards self-improving error diagnosis in multi-agent systems.

Armengol-Estapé, J., Carbonneaux, Q., Zhang, T., Markosyan, A. H., Seeker, V., Cummins, C., Kambadur, M., O'Boyle, M. F. P., Wang, S., Synnaeve, G., and Leather, H. J. (2025). What i cannot execute, i do not understand: Training and evaluating llms on program execution traces.

Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., and Ballas, N. (2023). Self-supervised learning from images with a joint-embedding predictive architecture.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. (2021). Program synthesis with large language models.

Baksys, M., Zetzsche, S., Bouissou, O., Delmas, R., Kong, S., and Holden, S. B. (2025). Atlas: Automated toolkit for large-scale verified code synthesis.

Barone, A. V. M. and Nok, P. T. (2026). Improving llm code reasoning via semantic equivalence self-play with formal verification.

Berkeley, U., AI, T., and Mila (2026). V1: Unifying generation and self-verification for parallel reasoners.

Bharti, V., Jha, S., Kumar, D., and Jalote, P. (2025). Loop invariant generation: A hybrid framework of reasoning optimised llms and smt solvers.

Bieber, D., Sutton, C., Larochelle, H., and Tarlow, D. (2020). Learning to execute programs with instruction pointer attention graph neural networks.

Bruce, J., Dennis, M., Edwards, A., Parker-Holder, J., Shi, Y., Hughes, E., Lai, M., Mavalankar, A., Steigerwald, R., Apps, C., Aytar, Y., Bechtle, S., Behbahani, F., Chan, S., Heess, N., Gonzalez, L., Osindero, S., Ozair, S., Reed, S., Zhang, J., Zolna, K., Clune, J., de Freitas, N., Singh, S., and Rocktäschel, T. (2024). Genie: Generative interactive environments.

Chae, H., Kim, N., iunn Ong, K. T., Gwak, M., Song, G., Kim, J., Kim, S., Lee, D., and Yeo, J. (2024). Web agents with world models: Learning and leveraging environment dynamics in web navigation.

Chen, J., Pan, Z., Hu, X., Li, Z., Li, G., and Xia, X. (2024). Reasoning runtime behavior of a program with llm: How far are we?

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. (2021). Evaluating large language models trained on code.

Chen, X., Lin, M., Schärli, N., and Zhou, D. (2023). Teaching large language models to self-debug.

Chen, Y., Liu, Y., Zhou, J., Hao, Y., Wang, J., Zhang, Y., Li, N., and Fan, C. (2025a). R1-code-interpreter: Llms reason with code via supervised and multi-stage reinforcement learning.

Chen, Z., Chang, K., Li, Z., Li, C., He, X., Chen, C., Wang, M., Xu, H., Han, Y., Li, H., and Wang, Y. (2025b). Chipseek: Optimizing verilog generation via eda-integrated reinforcement learning.

Da, J., Wang, C., Deng, X., Ma, Y., Barhate, N., and Hendryx, S. (2025). Agent-rlvr: Training software engineering agents via guidance and environment rewards.

Dainese, N., Merler, M., Alakuijala, M., and Marttinen, P. (2024). Generating code world models with large language models guided by monte carlo tree search.

DeepSeek-AI, Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., Liu, A., Xue, B., Wang, B., Wu, B., Feng, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Ding, H., Xin, H., Gao, H., Qu, H., Li, H., Guo, J., Li, J., Wang, J., Chen, J., Yuan, J., Qiu, J., Li, J., Cai, J. L., Ni, J., Liang, J., Chen, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Zhao, L., Wang, L., Zhang, L., Xu, L., Xia, L., Zhang, M., Zhang, M., Tang, M., Li, M., Wang, M., Li, M., Tian, N., Huang, P., Zhang, P., Wang, Q., Chen, Q., Du, Q., Ge, R., Zhang, R., Pan, R., Wang, R., Chen, R. J., Jin, R. L., Chen, R., Lu, S., Zhou, S., Chen, S., Ye, S., Wang, S., Yu, S., Zhou, S., Pan, S., Li, S. S., Zhou, S., Wu, S., Ye, S., Yun, T., Pei, T., Sun, T., Wang, T., Zeng, W., Zhao, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Xiao, W. L., An, W., Liu, X., Wang, X., Chen, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yang, X., Li, X., Su, X., Lin, X., Li, X. Q., Jin, X., Shen, X., Chen, X., Sun, X., Wang, X., Song, X., Zhou, X., Wang, X., Shan, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhang, Y., Xu, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Yu, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Ou, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Zhu, Y. X., Xu, Y., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Tang, Y., Zha, Y., Yan, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Ma, Z., Yan, Z., Wu, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Pan, Z., Huang, Z., Xu, Z., Zhang, Z., and Zhang, Z. (2025). Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning.

Ding, J., Zhang, Y., Shang, Y., Feng, J., Zhang, Y., Zong, Z., Yuan, Y., Su, H., Li, N., Piao, J., Deng, Y., Sukiennik, N., Gao, C., Xu, F., and Li, Y. (2024a). Understanding world or predicting future? a comprehensive survey of world models.

Ding, Y., Peng, J., Min, M. J., Kaiser, G., Yang, J., and Ray, B. (2024b). Semcoder: Training code language models with comprehensive semantics reasoning.

Ding, Y., Steenhoek, B., Pei, K., Kaiser, G., Le, W., and Ray, B. (2023). Traced: Execution-aware pre-training for source code.

Edwards, G. and Eslamimehr, M. (2026). Synergistic directed execution and llm-driven analysis for zero-day ai-generated malware detection.

FAIR CodeGen Team, Copet, J., Carbonneaux, Q., Cohen, G., Gehring, J., Kahn, J., Kossen, J., Kreuk, F., McMilin, E., Meyer, M., Wei, Y., Zhang, D., Zheng, K., Armengol-Estapé, J., Bashiri, P., Beck, M., Chambon, P., Charnalia, A., Cummins, C., Decugis, J., Fisches, Z. V., Fleuret, F., Gloeckle, F., Gu, A., Hassid, M., Haziza, D., Idrissi, B. Y., Keller, C., Kindi, R., Leather, H., Maimon, G., Markosyan, A., Massa, F., Mazaré, P.-E., Mella, V., Murray, N., Muzumdar, K., O'Hearn, P., Pagliardini, M., Pedchenko, D., Remez, T., Seeker, V., Selvi, M., Sultan, O., Wang, S., Wehrstedt, L., Yoran, O., Zhang, L., Cohen, T., Adi, Y., and Synnaeve, G. (2025). Cwm: An open-weights llm for research on code generation with world models.

Feng, X., Wan, Z., Wen, M., McAleer, S. M., Wen, Y., Zhang, W., and Wang, J. (2023). Alphazero-like tree-search can guide large language model decoding and training.

Gandhi, S., Tsay, J., Ganhotra, J., Kate, K., and Rizk, Y. (2025). When agents go astray: Course-correcting swe agents with prms.

Gehring, J., Zheng, K., Copet, J., Mella, V., Carbonneaux, Q., Cohen, T., and Synnaeve, G. (2024). Rlef: Grounding code llms in execution feedback with reinforcement learning.

Golubev, A., Trofimova, M., Polezhaev, S., Badertdinov, I., Nekrashevich, M., Shevtsov, A., Karasik, S., Abramov, S., Andriushchenko, A., Fisin, F., Skvortsov, S., and Yangel, B. (2025). Training long-context, multi-turn software engineering agents with reinforcement learning.

Gong, S., Zhang, R., Zheng, H., Gu, J., Jaitly, N., Kong, L., and Zhang, Y. (2025). Diffucoder: Understanding and improving masked diffusion models for code generation.

Gu, A., Rozière, B., Leather, H., Solar-Lezama, A., Synnaeve, G., and Wang, S. I. (2024a). Cruxeval: A benchmark for code reasoning, understanding and execution.

Gu, Y., Zhang, K., Ning, Y., Zheng, B., Gou, B., Xue, T., Chang, C., Srivastava, S., Xie, Y., Qi, P., Sun, H., and Su, Y. (2024b). Is your llm secretly a world model of the internet? model-based planning for web agents.

Guo, S., Domingues, O. D., Avalos, R., Courville, A., and Strub, F. (2025). World modelling improves language model agents.

Ha, D. and Schmidhuber, J. (2018). World models.

Hafner, D., Lillicrap, T., Ba, J., and Norouzi, M. (2019). Dream to control: Learning behaviors by latent imagination.

Hafner, D., Lillicrap, T., Norouzi, M., and Ba, J. (2020). Mastering atari with discrete world models.

Hafner, D., Pasukonis, J., Ba, J., and Lillicrap, T. (2023). Mastering diverse domains through world models.

Han, H., Xie, J., Ma, X., Zhu, W., Zhang, Z., Long, Z., Chen, H., and Ye, Q. (2026). Swe-trace: Optimizing long-horizon swe agents through rubric process reward models and heuristic test-time scaling.

Hao, S., Gu, Y., Ma, H., Hong, J. J., Wang, Z., Wang, D. Z., and Hu, Z. (2023). Reasoning with language model is planning with world model.

Huang, H., LeCun, Y., and Balestriero, R. (2025). Llm-jepa: Large language models meet joint embedding predictive architectures.

Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. (2024). Livecodebench: Holistic and contamination free evaluation of large language models for code.

Jia, C., Li, Z., Wang, P., Li, Y.-C., Hou, Z., Dong, Y., and Yu, Y. (2025). Controlling large language model with latent actions.

Jiang, J., Wang, F., Shen, J., Kim, S., and Kim, S. (2024). A survey on large language models for code generation.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. (2023). Swe-bench: Can language models resolve real-world github issues?

Jung, D., Zhou, W., and Chen, M. (2025). Code execution as grounded supervision for llm reasoning.

Kenneweg, T., Kenneweg, P., and Hammer, B. (2025). Jepa for rl: Investigating joint-embedding predictive architectures for reinforcement learning.

Khalifa, M., Agarwal, R., Logeswaran, L., Kim, J., Peng, H., Lee, M., Lee, H., and Wang, L. (2025). Process reward models that think.

Koh, J. Y., McAleer, S., Fried, D., and Salakhutdinov, R. (2024). Tree search for language model agents.

Lavon, B., Katz, S., and Wolf, L. (2025). Execution guided line-by-line code generation.

Li, S., Liu, Y., Du, S., Zeng, W., Xu, Z., Zhou, M., He, Y., Dong, H., Han, S., and Zhang, D. (2025a). Jupiter: Enhancing llm data analysis capabilities via notebook and inference-time value-guided search.

Li, X., He, X., Zhang, L., Wu, M., Li, X., and Liu, Y. (2025b). A comprehensive survey on world models for embodied ai.

Li, Y., Jiang, J., Zhang, T., Yang, C., Zhong, C., Yide, Y., Bin, L. W., Ouh, E. L., Shar, L. K., and Lo, D. (2026). An execution-verified multi-language benchmark for code semantic reasoning.

Li, Y., Meng, R., and Duck, G. J. (2025c). Large language model powered symbolic execution.

Liu, A. Z., Wang, X., Sansom, J., Fu, Y., Choi, J., Sohn, S., Kim, J., and Lee, H. (2024). Interactive and expressive code-augmented planning with large language models.

Liu, C., Lu, S., Chen, W., Jiang, D., Svyatkovskiy, A., Fu, S., Sundaresan, N., and Duan, N. (2023). Code execution with pre-trained language models.

Liu, Z., Ma, Y., Xu, J., Ai, J., Gao, X., Sun, H., and Roychoudhury, A. (2025). Agent that debugs: Dynamic state-guided vulnerability repair.

Maimon, G., Yoran, O., Kreuk, F., Hassid, M., Cohen, G., Chambon, P., and Adi, Y. (2026). Self-execution simulation improves coding models.

Mitchell, J. L., Kim, B. H., Zhou, C., and Wang, C. (2025). Understanding formal reasoning failures in llms as abstract interpreters.

Monperrus, M. (2026). Bootstrapping coding agents: The specification is the program.

Ni, A., Allamanis, M., Cohan, A., Deng, Y., Shi, K., Sutton, C., and Yin, P. (2024). Next: Teaching large language models to reason about code execution.

Ni, A., Iyer, S., Radev, D., Stoyanov, V., tau Yih, W., Wang, S. I., and Lin, X. V. (2023). Lever: Learning to verify language-to-code generation with execution.

Nye, M., Andreassen, A. J., Gur-Ari, G., Michalewski, H., Austin, J., Bieber, D., Dohan, D., Lewkowycz, A., Bosma, M., Luan, D., Sutton, C., and Odena, A. (2021). Show your work: Scratchpads for intermediate computation with language models.

Olausson, T. X., Inala, J. P., Wang, C., Gao, J., and Solar-Lezama, A. (2023). Is self-repair a silver bullet for code generation?

Ouyang, S., Yu, W., Ma, K., Xiao, Z., Zhang, Z., Jia, M., Han, J., Zhang, H., and Yu, D. (2024). Repograph: Enhancing ai software engineering with repository-level code graph.

Pan, J., Wang, X., Neubig, G., Jaitly, N., Ji, H., Suhr, A., and Zhang, Y. (2024). Training software engineering agents and verifiers with swe-gym.

Pink, M., Wu, Q., Vo, V. A., Turek, J., Mu, J., Huth, A., and Toneva, M. (2025). Position: Episodic memory is the missing piece for long-term llm agents.

Pourcel, J., Colas, C., and Oudeyer, P.-Y. (2025). Self-improving language models for evolutionary program synthesis: A case study on arc-agi.

Pu, Y., Niu, Y., Yang, Z., Ren, J., Li, H., and Liu, Y. (2024). Unizero: Generalized and efficient planning with scalable latent world models.

Qiu, Z., Qiao, S., Xu, K., Zhu, Y., Du, L., Zhang, N., and Chen, H. (2026). Rewarding the scientific process: Process-level reward modeling for agentic data analysis.

Rahmani, B. (2026). Debugging code world models.

Reed, S. and de Freitas, N. (2015). Neural programmer-interpreters.

Research, S. A. et al. (2026). Agent world model: Infinity synthetic environments for agentic reinforcement learning.

Ribeiro, F., Spiess, C., Devanbu, P., and Nadi, S. (2025). On llms' internal representation of code correctness.

Richens, J., Abel, D., Bellot, A., and Everitt, T. (2025). General agents contain world models.

Robeyns, M., Szummer, M., and Aitchison, L. (2025). A self-improving coding agent.

Rodionov, S. (2026). Executable world models for arc-agi-3 in the era of coding agents.

Sahoo, S. (2025). The double life of code world models: Provably unmasking malicious behavior through execution traces.

Samadi, M., Ficek, A., Narenthiran, S., Jain, S., Ahmad, W. U., Majumdar, S., Noroozi, V., and Ginsburg, B. (2025). Scaling test-time compute to achieve ioi gold medal with open-weight models.

Schultz, J., Adamek, J., Jusup, M., Lanctot, M., Kaisers, M., Perrin, S., Hennes, D., Shar, J., Lewis, C., Ruoss, A., Zahavy, T., Veličković, P., Prince, L., Singh, S., Malmi, E., and Tomašev, N. (2024). Mastering board games by external and internal planning with language models.

Shi, Z., Swersky, K., Tarlow, D., Ranganathan, P., and Hashemi, M. (2019). Learning execution through neural code fusion.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning.

Singh, G., Guha, A., Kailkhura, B., and Menon, H. (2026). Learning reasoning world models for parallel code.

Sun, C., Sun, Y., Amrollahi, D., Zhang, E., Lahiri, S., Lu, S., Dill, D., and Barrett, C. (2025). Veristruct: Ai-assisted automated verification of data-structure modules in verus.

Sun, S., Song, H., Huang, L., Jiang, J., Le, R., Lv, Z., Chen, Z., Hu, Y., Luo, W., Zhao, W. X., Song, Y., Xu, H., Zhang, T., and Wen, J.-R. (2026). Swe-world: Building software engineering agents in docker-free environments.

Tahimic, K. and Cheng, C. (2025). Mechanistic interpretability of code correctness in llms via sparse autoencoders.

Tan, H., Li, W., Tian, X., Wang, S., Liu, J., Li, J., and Zhang, Y. (2025). Sk2decompile: Llm-based two-phase binary decompilation from skeleton to skin.

Tang, L., Ye, H., Chu, Z., Ye, M., Liu, Z., Ren, X., and Bao, L. (2026). Execverify: White-box rl with verifiable stepwise rewards for code execution reasoning.

Team, Q. (2026). Scaling agentic verifier for competitive coding.

Technologies, H. et al. (2026). Cli-gym: Scalable cli task generation via agentic environment inversion.

Teng, F., Pan, M., Zhang, X., He, Z., Yang, Y., Chai, X., Qi, M., Lu, L., and Yin, J. (2025). Verirl: Boosting the llm-based verilog code generation via reinforcement learning.

Thakur, A., Lee, J., Tsoukalas, G., Sistla, M., Zhao, M., Zetzsche, S., Durrett, G., Yue, Y., and Chaudhuri, S. (2025). Clever: A curated benchmark for formally verified code generation.

Thimmaiah, A., Zhang, J., Srinivasa, J., Li, J. J., and Gligoric, M. (2025). Plsemanticsbench: Large language models as programming language interpreters.

Tu, H., Zhao, H., Song, Y., Zafar, M., Meng, R., and Roychoudhury, A. (2025). Agentic verification of software systems.

Ugare, S. and Chandra, S. (2026). Agentic code reasoning.

Wang, J., Xie, X., Hu, Q., Liu, S., and Li, Y. (2025a). Do code semantics help? a comprehensive study on execution trace-based information for code large language models.

Wang, K., Singh, R., and Su, Z. (2017). Dynamic neural program embedding for program repair.

Wang, W., Liu, K., Chen, A. R., Li, G., Jin, Z., Huang, G., and Ma, L. (2024a). Python symbolic execution with llm-powered code generation.

Wang, W., Piękos, P., Nanbo, L., Laakom, F., Chen, Y., Ostaszewski, M., Zhuge, M., and Schmidhuber, J. (2025b). Huxley-gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine.

Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., and Ji, H. (2024b). Executable code actions elicit better llm agents.

Wang, Y., Xu, X., Zhu, X., Gu, X., and Shen, B. (2025c). Salt4decompile: Inferring source-level abstract logic tree for llm-based binary decompilation.

Wang, Y., Zhang, Y., Li, G., Zhi, C., Li, B., Huang, F., Li, Y., and Deng, S. (2025d). Inspectcoder: Dynamic analysis-enabled self repair through interactive llm-debugger collaboration.

Wei, A., Cao, J., Li, R., Chen, H., Zhang, Y., Wang, Z., Liu, Y., Teixeira, T. S. F. X., Yang, D., Wang, K., and Aiken, A. (2025a). Equibench: Benchmarking large language models' reasoning about program semantics via equivalence checking.

Wei, A., Suresh, T., Cao, J., Kannan, N., Wu, Y., Yan, K., Teixeira, T. S. F. X., Wang, K., and Aiken, A. (2025b). Codearc: Benchmarking reasoning capabilities of llm agents for inductive program synthesis.

Wei, A., Tan, H., Suresh, T., Mendoza, D., Teixeira, T. S. F. X., Wang, K., Trippel, C., and Aiken, A. (2025c). Vericoder: Enhancing llm-based rtl code generation through functional correctness validation.

Wei, Y., Duchenne, O., Copet, J., Carbonneaux, Q., Zhang, L., Fried, D., Synnaeve, G., Singh, R., and Wang, S. I. (2025d). Swe-rl: Advancing llm reasoning via reinforcement learning on open software evolution.

Wei, Y., Sun, Z., McMilin, E., Gehring, J., Zhang, D., Synnaeve, G., Fried, D., Zhang, L., and Wang, S. (2025e). Toward training superintelligent software agents through self-play swe-rl.

Xia, C. S., Deng, Y., Dunn, S., and Zhang, L. (2024). Agentless: Demystifying llm-based software engineering agents.

Xie, Z., Ye, J., Zheng, L., Gao, J., Dong, J., Wu, Z., Zhao, X., Gong, S., Jiang, X., Li, Z., and Kong, L. (2025). Dream-coder 7b: An open diffusion language model for code.

Xu, R., Cao, J., Lu, Y., Wen, M., Lin, H., Han, X., He, B., Cheung, S.-C., and Sun, L. (2024). Cruxeval-x: A benchmark for multilingual code reasoning, understanding and execution.

Yan, C., Che, F., Huang, X., Xu, X., Li, X., Li, Y., Qu, X., Shi, J., Lin, C., Yang, Y., Yuan, B., Zhao, H., Qiao, Y., Zhou, B., and Fu, J. (2025). Re:form – reducing human priors in scalable formal software verification with rl in llms: A preliminary study on dafny.

Yan, S., Wang, S., Duan, Y., Hong, H., Lee, K., Kim, D., and Hong, Y. (2024). An llm-assisted easy-to-trigger backdoor attack on code completion models: Injecting disguised vulnerabilities against strong detection.

Yang, F., Ma, X., Wang, S., Xu, X., Cao, Q., Zhan, N., Li, X., and Gu, B. (2025). Integrating symbolic execution with llms for automated generation of program specifications.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. (2024). Swe-agent: Agent-computer interfaces enable automated software engineering.

Yang, J., Lieret, K., Ma, J., Thakkar, P., Pedchenko, D., Sootla, S., McMilin, E., Yin, P., Hou, R., Synnaeve, G., Yang, D., and Press, O. (2026a). Programbench: Can language models rebuild programs from scratch?

Yang, J., Zhang, W., Wu, J., Cheng, J., Zheng, T., Xu, F., Gu, W., Jing, L., Du, Y., Li, J., Li, Y., Xing, Y., Hao, C., Tao, R., Gong, R., Liu, A., Li, Z., Tang, M., Lin, C., Chen, S., Zhao, W. X., Liu, X., Zhou, M., Dai, B., and Lv, W. (2026b). Incoder-32b-thinking: Industrial code world model for thinking.

Yang, S. (2026). World models as an intermediary between agents and the real world.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., and Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models.

Yu, X., Peng, B., Xu, R., Galley, M., Cheng, H., Nath, S., Gao, J., and Yu, Z. (2025). Dyna-think: Synergizing reasoning, acting, and world model simulation in ai agents.

Yu, X., Peng, B., Xu, R., Shen, Y., He, P., Nath, S., Singh, N., Gao, J., and Yu, Z. (2026). Reinforcement world model learning for llm-based agents.

Zaremba, W. and Sutskever, I. (2014). Learning to execute.

Zeng, Z., Zhang, Y., Shan, Y., Hua, K., Fang, S., Liu, Z., Liu, J., Wang, H., Zheng, Y., Ding, M., Shen, K., Zhang, G., Huang, W., and Qiu, X. (2026). Understanding by reconstruction: Reversing the software development process for llm pretraining.

Zhang, J., Hu, S., Lu, C., Lange, R., and Clune, J. (2025a). Darwin godel machine: Open-ended evolution of self-improving agents.

Zhang, R., Qin, P., Cao, Q., Xue, E., and Xie, P. (2026). Funprm: Function-as-step process reward model with meta reward correction for code generation.

Zhang, Y., Pan, Y., Wang, Y., and Cai, J. (2024a). Pybench: Evaluating llm agent on various real-world coding tasks.

Zhang, Y., Ruan, H., Fan, Z., and Roychoudhury, A. (2024b). Autocoderover: Autonomous program improvement.

Zhang, Y., Shu, J., Ma, Y., Lin, X., Wu, S., and Sang, J. (2025b). Memory as action: Autonomous context curation for long-horizon agentic tasks.

Zhang, Y., Wu, S., Yang, Y., Shu, J., Xiao, J., Kong, C., and Sang, J. (2024c). o1-coder: an o1 replication for coding.

Zhuge, M., Zhao, C., Liu, H., Zhou, Z., Liu, S., Wang, W., Chang, E., Lan, G. L., Fei, J., Zhang, W., Sun, Y., Cai, Z., Liu, Z., Xiong, Y., Yang, Y., Tian, Y., Shi, Y., Chandra, V., and Schmidhuber, J. (2026). Neural computers.